

# Homework 01

## Table of contents

|                                                                       |    |
|-----------------------------------------------------------------------|----|
| Question 1 (Least squares and matrix calculations) .....              | 1  |
| Answer .....                                                          | 2  |
| Question 2 (Gaussian likelihood for linear regression) .....          | 3  |
| Answer .....                                                          | 4  |
| Question 3 (Starting values in nonconvex optimization) .....          | 6  |
| Answer .....                                                          | 7  |
| Question 4 (Nearly collinear predictors) .....                        | 9  |
| Answer .....                                                          | 10 |
| Question 5 (Data preparation and summary statistics) .....            | 13 |
| Answer .....                                                          | 13 |
| Question 6 (Can 39 million Fitbit records represent US adults?) ..... | 16 |
| Source .....                                                          | 17 |
| Answer .....                                                          | 17 |

## Question 1 (Least squares and matrix calculations)

Set the random seed to 43201. Generate  $n = 100$  observations with  $p = 5$  predictors according to

$$x_{ij} \stackrel{\text{iid}}{\sim} N(0, 1), \quad \varepsilon_i \stackrel{\text{iid}}{\sim} N(0, 0.7^2),$$

with all predictors and errors generated independently. Let

$$\mathbf{X} = [\mathbf{1} \ \mathbf{x}_1 \ \cdots \ \mathbf{x}_5], \quad \boldsymbol{\beta} = (3, 1.4, -0.9, 0.7, 0, 1.1)^T,$$

and generate

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}.$$

- Generate the data and report the dimensions and rank of  $\mathbf{X}$ . Calculate the least-squares estimate  $\hat{\boldsymbol{\beta}}$  using a numerically stable least-squares routine, without explicitly calculating  $(\mathbf{X}^T \mathbf{X})^{-1}$ . Compare the estimates with the coefficients used to generate the data.
- Calculate the fitted values, the residual vector  $\mathbf{r} = \mathbf{y} - \mathbf{X}\hat{\boldsymbol{\beta}}$ , and the training root mean squared error

$$\text{RMSE} = \left( \frac{1}{n} \sum_{i=1}^n r_i^2 \right)^{1/2}.$$

- Report  $\|\mathbf{X}^T \mathbf{r}\|_{\infty}$ . Explain why this value should be close to zero and what it tells us about the residual vector.

## Answer

```
set.seed(43201)
n <- 100
p <- 5

X_pred <- matrix(rnorm(n * p), nrow = n, ncol = p)
colnames(X_pred) <- paste0("x", 1:p)
eps <- rnorm(n, mean = 0, sd = 0.7)

X <- cbind(Intercept = 1, X_pred)
beta <- c(3, 1.4, -0.9, 0.7, 0, 1.1)
y <- as.vector(X %*% beta + eps)
```

### (a) Dimensions, rank, and the least-squares estimate.

```
dim(X)
```

```
[1] 100  6
```

```
qrX <- qr(X)
qrX$rank
```

```
[1] 6
```

```
# QR-based solve: no explicit (X'X)^{-1} is ever formed
beta_hat <- qr.coef(qrX, y)
```

```
data.frame(
  true      = beta,
  estimate  = as.vector(beta_hat),
  error     = as.vector(beta_hat) - beta,
  row.names = colnames(X)
)
```

|           | true | estimate   | error      |
|-----------|------|------------|------------|
| Intercept | 3.0  | 3.0063210  | 0.00632099 |
| x1        | 1.4  | 1.4171604  | 0.01716042 |
| x2        | -0.9 | -0.8883453 | 0.01165468 |
| x3        | 0.7  | 0.7952834  | 0.09528339 |
| x4        | 0.0  | 0.0484378  | 0.04843780 |
| x5        | 1.1  | 1.1319341  | 0.03193413 |

The design matrix is  $100 \times 6$  with rank 6, so it has full column rank and the least-squares solution is unique. Every estimate lands near the coefficient that generated the data. The one worth singling out is  $x_4$ : its true coefficient is exactly 0, but it is estimated as roughly 0.05. With  $n = 100$  and  $\sigma = 0.7$  the standard error of a slope here is about 0.07, so an estimate of that size is ordinary sampling noise rather than evidence of a real effect. Least squares never returns exactly zero for an irrelevant predictor.

**(b) Fitted values, residuals, and training RMSE.**

```
fitted_vals <- as.vector(X %*% beta_hat)
r <- y - fitted_vals

rmse <- sqrt(mean(r^2))
rmse
```

```
[1] 0.6763696
```

The training RMSE is about 0.676, slightly below the true error standard deviation  $\sigma = 0.7$ . This is not a coincidence. The residuals are computed from the same 100 observations used to fit the six coefficients, so the fit absorbs part of the noise and the training error understates the true noise level. Question 2(b) makes this precise: dividing by  $n$  rather than  $n - p - 1$  gives a downward-biased estimate of  $\sigma^2$ .

**(c) Orthogonality of the residual vector.**

```
max(abs(t(X) %*% r))
```

```
[1] 8.915091e-14
```

The value is on the order of  $10^{-13}$ , which is zero to machine precision once compared against the natural scale of the problem (entries of  $\mathbf{X}^T \mathbf{y}$  are of order  $10^2$ ).

It should be zero because  $\hat{\beta}$  solves the normal equations

$$\mathbf{X}^T (\mathbf{y} - \mathbf{X}\hat{\beta}) = \mathbf{X}^T \mathbf{r} = \mathbf{0}.$$

The residual vector is therefore orthogonal to every column of  $\mathbf{X}$ , and so to the entire column space. Equivalently, the fitted values are the orthogonal projection of  $\mathbf{y}$  onto that column space, and  $\mathbf{r}$  is the component of  $\mathbf{y}$  lying outside it. Because the first column is the intercept  $\mathbf{1}$ , orthogonality also forces  $\sum_i r_i = 0$ : the residuals have mean zero. The tiny nonzero value that remains is floating-point rounding, not a failure of the fit.

## Question 2 (Gaussian likelihood for linear regression)

Continue with the data from Question 1. Suppose

$$\mathbf{Y} \mid \mathbf{X} \sim N_n(\mathbf{X}\beta, \sigma^2 \mathbf{I}_n).$$

The log-likelihood, including the constant term, is

$$\ell(\beta, \sigma^2) = -\frac{n}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} (\mathbf{y} - \mathbf{X}\beta)^T (\mathbf{y} - \mathbf{X}\beta).$$

- For fixed  $\sigma^2$ , explain why maximizing  $\ell(\beta, \sigma^2)$  over  $\beta$  is equivalent to minimizing the residual sum of squares. What does this imply about the maximum-likelihood estimate of  $\beta$ ?
- For fixed  $\beta$ , differentiate the log-likelihood with respect to  $\sigma^2$  and derive its maximum-likelihood estimate. Calculate this estimate at  $\hat{\beta}$  and compare it with the unbiased estimate of  $\sigma^2$ . Explain why the two denominators differ.
- Let

$$\mathbf{e}_{x_2} = (0, 0, 1, 0, 0, 0)^T.$$

Using the variance estimate from part b, plot

$$\ell(\hat{\beta} + t\mathbf{e}_{x_2}, \hat{\sigma}_{\text{MLE}}^2)$$

over a grid of  $t$  values from  $-1.25$  to  $1.25$ . State where the maximum occurs and explain why this agrees with the least-squares result.

## Answer

### (a) Why maximizing the likelihood over $\beta$ is least squares.

Write the log-likelihood as

$$\ell(\beta, \sigma^2) = \underbrace{-\frac{n}{2} \log(2\pi\sigma^2)}_{\text{free of } \beta} - \frac{1}{2\sigma^2} \underbrace{(\mathbf{y} - \mathbf{X}\beta)^T (\mathbf{y} - \mathbf{X}\beta)}_{\text{RSS}(\beta)}.$$

With  $\sigma^2$  held fixed, the first term is a constant in  $\beta$  and the multiplier  $1/(2\sigma^2)$  is a fixed positive number. Maximizing  $\ell$  over  $\beta$  is therefore the same as maximizing  $-\text{RSS}(\beta)$ , which is the same as minimizing  $\text{RSS}(\beta)$ .

The implication is that under the Gaussian model the maximum-likelihood estimate of  $\beta$  is the least-squares estimate,  $\hat{\beta}_{\text{MLE}} = \hat{\beta}_{\text{OLS}}$ , and it does not depend on the value of  $\sigma^2$ . Least squares is not merely a convenient geometric criterion here; it is what the normal likelihood implies.

### (b) The MLE of $\sigma^2$ .

Treat  $s = \sigma^2$  as the variable and differentiate at fixed  $\beta$ :

$$\frac{\partial \ell}{\partial s} = -\frac{n}{2s} + \frac{\text{RSS}(\beta)}{2s^2}.$$

Setting this to zero gives  $ns = \text{RSS}(\beta)$ , so

$$\hat{\sigma}_{\text{MLE}}^2 = \frac{\text{RSS}(\beta)}{n}.$$

This is a maximum: for  $s$  below the solution the derivative is positive and for  $s$  above it the derivative is negative, so  $\ell$  increases then decreases.

```

RSS <- sum(r^2)
k <- ncol(X)          # 6 estimated coefficients (intercept + 5 slopes)

s2_mle <- RSS / n
s2_unb <- RSS / (n - k)

c(RSS = RSS, s2_mle = s2_mle, s2_unbiased = s2_unb, true_sigma2 = 0.7^2)

```

| RSS        | s2_mle    | s2_unbiased | true_sigma2 |
|------------|-----------|-------------|-------------|
| 45.7475864 | 0.4574759 | 0.4866765   | 0.4900000   |

```

s2_unb / s2_mle          # equals n / (n - k)

```

```

[1] 1.06383

```

Evaluated at  $\hat{\beta}$  the MLE is 0.4575, while the unbiased estimate  $RSS/(n - k) = 0.4867$  sits much closer to the true  $\sigma^2 = 0.49$ .

The denominators differ because the residuals are not the true errors. Fitting  $k = 6$  coefficients forces  $\mathbf{r}$  to be orthogonal to all six columns of  $\mathbf{X}$ , so  $\mathbf{r}$  is confined to an  $(n - k)$ -dimensional subspace and carries only  $n - k = 94$  free degrees of freedom rather than  $n = 100$ . This makes RSS systematically smaller than the sum of squares of the true errors, and in expectation  $E[RSS] = (n - k)\sigma^2$ . Dividing by  $n - k$  therefore gives an unbiased estimate, while dividing by  $n$  underestimates  $\sigma^2$  by the factor  $(n - k)/n$ . The MLE is biased downward because maximum likelihood does not correct for the parameters consumed in estimating the mean.

### (c) Profiling the log-likelihood along $\mathbf{e}_{x_2}$ .

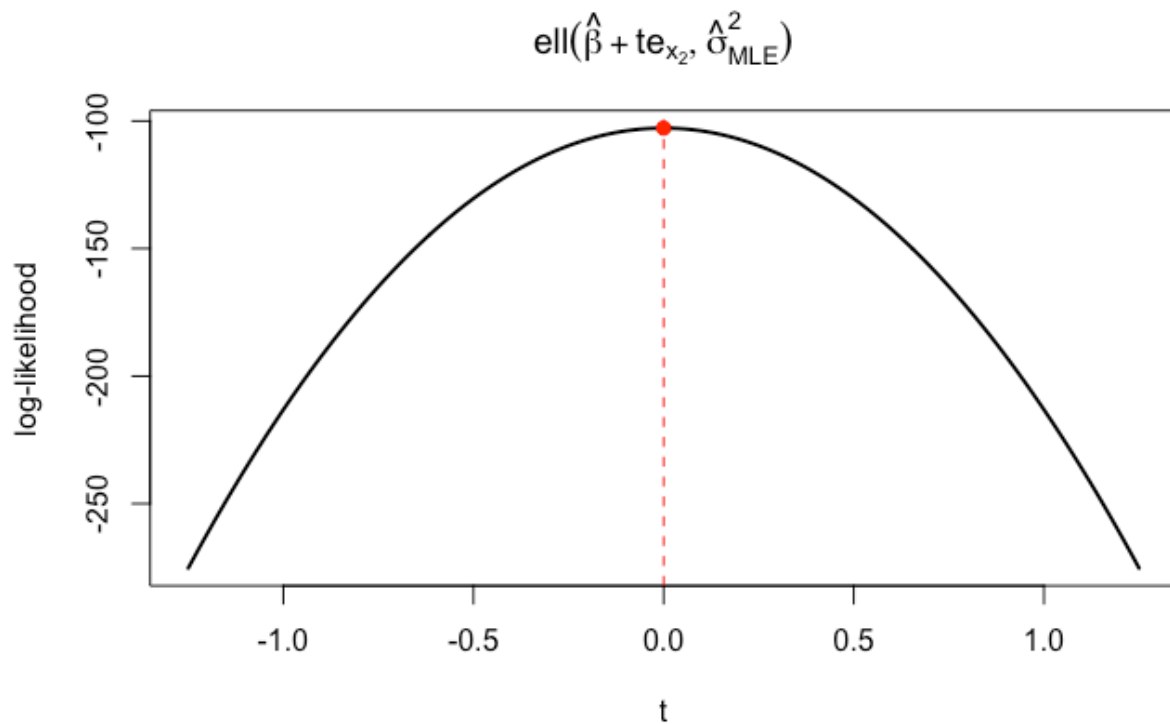
```

loglik <- function(b, s2) {
  -n / 2 * log(2 * pi * s2) - sum((y - X %*% b)^2) / (2 * s2)
}

e_x2 <- c(0, 0, 1, 0, 0, 0)          # perturbs the x2 coefficient only
t_grid <- seq(-1.25, 1.25, length.out = 501)
ll <- sapply(t_grid, function(t) loglik(beta_hat + t * e_x2, s2_mle))

plot(t_grid, ll, type = "l", lwd = 2,
     xlab = expression(t), ylab = "log-likelihood",
     main = expression(ell(hat(beta) + t*e[x[2]], hat(sigma)[MLE]^2)))
abline(v = 0, lty = 2, col = "red")
points(0, loglik(beta_hat, s2_mle), pch = 19, col = "red")

```



```
t_grid[which.max(ll)] # location of the maximum
```

```
[1] 0
```

The maximum occurs at  $t = 0$ , that is, exactly at  $\hat{\beta}$  itself, and the curve is symmetric about that point.

This is precisely what part (a) predicts. Since  $\sigma^2$  is held fixed, the profile is a strictly decreasing function of RSS, and  $\hat{\beta}$  is the global minimizer of RSS, so any move away from it in any direction can only lower the log-likelihood. The shape is also exactly quadratic, not merely bowl-shaped. Because  $\mathbf{r} \perp \mathbf{X}\mathbf{e}_{x_2}$ , the cross term vanishes and

$$\text{RSS}(\hat{\beta} + t \mathbf{e}_{x_2}) = \text{RSS}(\hat{\beta}) + t^2 \|\mathbf{X}\mathbf{e}_{x_2}\|^2,$$

so the log-likelihood falls off as  $-t^2 \|\mathbf{X}\mathbf{e}_{x_2}\|^2 / (2\hat{\sigma}_{\text{MLE}}^2)$ . Here  $\|\mathbf{X}\mathbf{e}_{x_2}\|^2 \approx 101$ , and the curvature is steep: shifting the  $x_2$  coefficient by a single unit costs about 110 log-likelihood units.

### Question 3 (Starting values in nonconvex optimization)

Consider the function

$$f(x) = \exp(1.5x) - 3(x + 6)^2 - 0.05x^3,$$

with derivative

$$f'(x) = 1.5 \exp(1.5x) - 6(x + 6) - 0.15x^2.$$

- Plot  $f(x)$  over the interval  $[-40, 7]$ . Based on the plot, describe the important features of the objective function that may affect numerical optimization.
- Use BFGS to minimize  $f(x)$  twice, starting at  $x^{(0)} = -15$  and  $x^{(0)} = 0$ . Supply  $f'(x)$  to the optimizer. For each run, report the final value of  $x$ , the final objective value,  $|f'(x)|$ , and whether the optimizer reported convergence.
- Explain why the two runs can converge to different answers. Which run gives the lower objective value? What additional evidence would be needed before claiming that this point is the global minimum?

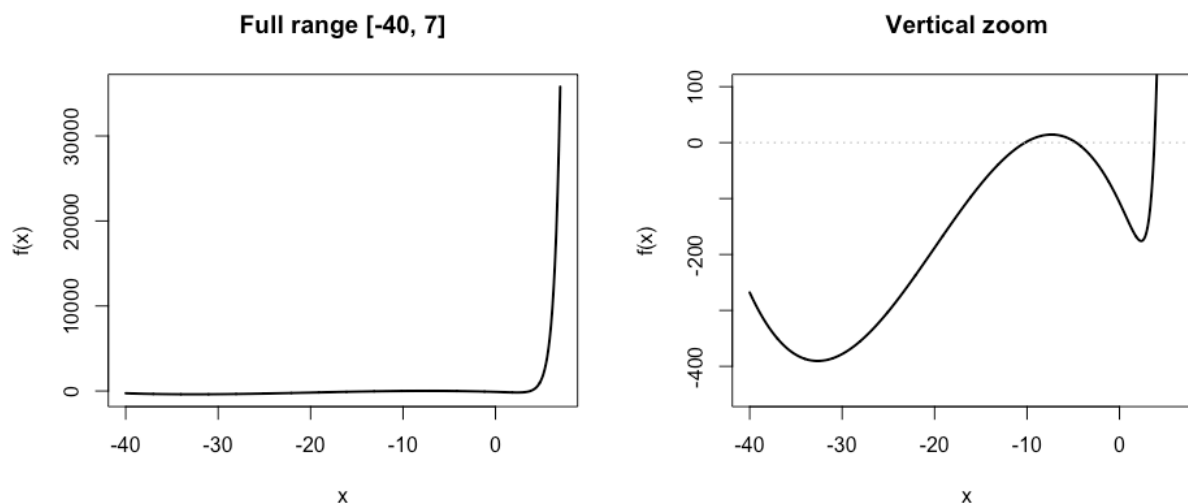
## Answer

### (a) The shape of the objective.

```
f_obj <- function(x) exp(1.5 * x) - 3 * (x + 6)^2 - 0.05 * x^3
f_grad <- function(x) 1.5 * exp(1.5 * x) - 6 * (x + 6) - 0.15 * x^2

xs <- seq(-40, 7, length.out = 2000)

par(mfrow = c(1, 2))
plot(xs, f_obj(xs), type = "l", lwd = 2,
     xlab = "x", ylab = "f(x)", main = "Full range [-40, 7]")
plot(xs, f_obj(xs), type = "l", lwd = 2, ylim = c(-450, 100),
     xlab = "x", ylab = "f(x)", main = "Vertical zoom")
abline(h = 0, col = "grey70", lty = 3)
```



```
par(mfrow = c(1, 1))
```

Several features matter for optimization.

The function is **not convex**. It has two separate local minima, near  $x \approx -32.6$  and  $x \approx 2.35$ , separated by a local maximum near  $x \approx -7.35$ . A local method started anywhere in the interval will descend into whichever basin contains its starting point and stop there.

The **scale is wildly uneven**. On  $[-40, 7]$  the function ranges from about  $-390$  to about  $35,791$ , and nearly all of that range comes from the exponential term over the last two units on the right. The left panel is dominated by that spike; a vertical zoom is required before the interesting structure is visible at all. Gradients differ by orders of magnitude across the domain, so a step length that is sensible in one region is badly wrong in another.

The **exponential term makes the right side explosive**. For  $x \gtrsim 4$  the term  $\exp(1.5x)$  dominates completely, and both  $f$  and  $f'$  grow extremely fast. A line search that steps too far to the right can produce enormous values and, further out, numerical overflow.

The **cubic term controls the far left**. As  $x \rightarrow -\infty$  the term  $-0.05x^3$  tends to  $+\infty$ , so the function eventually turns back upward. The left local minimum is a genuine basin rather than a slope running off to  $-\infty$ , but it sits far from the origin, so a poorly placed starting value can be a long way from it.

### (b) Two BFGS runs.

```
run_bfgs <- function(x0) {
  o <- optim(par = x0, fn = f_obj, gr = f_grad, method = "BFGS")
  data.frame(
    start      = x0,
    final_x    = o$par,
    final_f    = o$value,
    abs_grad   = abs(f_grad(o$par)),
    convergence = o$convergence,
    converged  = o$convergence == 0
  )
}

results <- rbind(run_bfgs(-15), run_bfgs(0))
results
```

|   | start | final_x    | final_f   | abs_grad     | convergence | converged |
|---|-------|------------|-----------|--------------|-------------|-----------|
| 1 | -15   | -32.649111 | -390.3858 | 8.725087e-08 | 0           | TRUE      |
| 2 | 0     | 2.349967   | -175.8626 | 6.556057e-07 | 0           | TRUE      |

| start | final $x$ | $f(x)$   | $ f'(x) $            | convergence code |
|-------|-----------|----------|----------------------|------------------|
| -15   | -32.6491  | -390.386 | $8.7 \times 10^{-8}$ | 0 (success)      |
| 0     | 2.3500    | -175.863 | $6.6 \times 10^{-7}$ | 0 (success)      |

Both runs report `convergence = 0`, and in both the gradient is essentially zero at the returned point. Each run genuinely succeeded at the task it was given. They simply succeeded at different places.

### (c) Why the runs disagree.

BFGS is a **local descent method**. From its starting value it repeatedly moves in a descent direction, so it can never climb over the local maximum at  $x \approx -7.35$  that separates the two basins. The starting value alone determines which basin it is trapped in. At  $x^{(0)} = -15$  the derivative is  $f'(-15) \approx +20.3$ ; descending means moving left, and the run settles into the left basin. At  $x^{(0)} = 0$  the derivative is  $f'(0) = -34.5$ ; descending means moving right, and the run settles into the right basin. The stopping rule only checks that the gradient is near zero, which is a statement about the immediate neighborhood and says nothing about the rest of the domain.

The run from  $x^{(0)} = -15$  gives the **lower objective value**,  $-390.386$  against  $-175.863$ .

A zero gradient and a success flag are not evidence of global optimality. Before claiming a global minimum one would want, at minimum: many restarts from starting values spread across the domain, all converging to the same point or to nothing lower; a check of the boundary or limiting behavior, since a minimum can sit at an endpoint rather than at a stationary point; and confirmation that the stationary point is a minimum rather than a saddle or maximum, via  $f''$ .

For this particular function the claim can be settled completely, which is unusual:

```
# every stationary point of f, located by bracketing the roots of f'
brackets <- list(c(-40, -20), c(-20, -3), c(0, 5))
stat_pts <- sapply(brackets, function(b) uniroot(f_grad, b, tol = 1e-12)$root)

# second derivative decides minimum vs maximum
f_hess <- function(x) 2.25 * exp(1.5 * x) - 0.3 * x - 6

data.frame(
  x      = stat_pts,
  f      = f_obj(stat_pts),
  fpp    = f_hess(stat_pts),
  type   = ifelse(f_hess(stat_pts) > 0, "minimum", "maximum")
)
```

|   | x          | f          | fpp       | type    |
|---|------------|------------|-----------|---------|
| 1 | -32.649111 | -390.38577 | 3.794733  | minimum |
| 2 | -7.350883  | 14.38579   | -3.794699 | maximum |
| 3 | 2.349967   | -175.86262 | 69.687243 | minimum |

Since  $f''(x) = 2.25e^{1.5x} - 0.3x - 6$  changes sign exactly twice,  $f'$  is monotone on three intervals and so has at most three roots. All three are found above, so there are no other stationary points anywhere on the real line. Because  $f(x) \rightarrow +\infty$  in both directions, the global minimum must be one of the two local minima, and it is the left one at  $x \approx -32.649$ . The run started at  $x^{(0)} = -15$  found the global minimum; the run started at  $x^{(0)} = 0$  did not. Nothing in either run's output distinguishes the two cases.

## Question 4 (Nearly collinear predictors)

Continue with  $\mathbf{X}$  and  $\mathbf{y}$  from Question 1. Set the random seed to 43202 and generate

$$u_i \stackrel{\text{iid}}{\sim} N(0, 1), \quad i = 1, \dots, n.$$

Define

$$\mathbf{x}_6 = \mathbf{x}_1 + 10^{-4}\mathbf{u}, \quad \mathbf{X}_+ = [\mathbf{X} \ \mathbf{x}_6],$$

and let

$$\mathbf{y}^* = \mathbf{y} + 10^{-3}\mathbf{u}.$$

Use a numerically stable least-squares routine throughout.

- Fit  $\mathbf{y}$  using  $\mathbf{X}$  and  $\mathbf{X}_+$ . For each design matrix, report the condition number and training RMSE. For the augmented fit, also report the coefficients of  $\mathbf{x}_1$  and  $\mathbf{x}_6$ .
- Fit  $\mathbf{y}^*$  using  $\mathbf{X}_+$ . Report the changes in the coefficients of  $\mathbf{x}_1$  and  $\mathbf{x}_6$ . Compare the two fitted-value vectors using their root mean squared difference and maximum absolute difference.
- Use the identity

$$\mathbf{y}^* - \mathbf{y} = 10(\mathbf{x}_6 - \mathbf{x}_1)$$

to explain why the fitted values change very little while the two coefficients change substantially. What does this example suggest about interpreting separate effects for nearly collinear predictors?

## Answer

```
set.seed(43202)
u <- rnorm(n)
x6 <- X[, "x1"] + 1e-4 * u          # a near-copy of x1
Xp <- cbind(X, x6 = x6)            # augmented design, 100 x 7
ystar <- y + 1e-3 * u              # slightly perturbed response

cond_num <- function(M) { d <- svd(M)$d; d[1] / d[length(d)] }
rmse_of <- function(v) sqrt(mean(v^2))
```

### (a) Condition numbers and training fit.

```
b_X <- qr.coef(qr(X), y)
b_Xp <- qr.coef(qr(Xp), y)

data.frame(
  design      = c("X", "X_plus"),
  condition   = c(cond_num(X), cond_num(Xp)),
  train_rmse  = c(rmse_of(y - X %*% b_X), rmse_of(y - Xp %*% b_Xp))
)
```

```
  design    condition train_rmse
1      X      1.524914  0.6763696
2 X_plus 20209.203501  0.6754668
```

```
# coefficients on the two nearly identical columns
b_Xp[c("x1", "x6")]
```

```
      x1      x6
351.6766 -350.2588
```

```
sum(b_Xp[c("x1", "x6")])      # compare with the x1 coefficient from the clean fit
```

```
[1] 1.41785
```

```
b_X["x1"]
```

```
      x1
1.41716
```

The condition number rises from about 1.5 for  $\mathbf{X}$  to about  $2.0 \times 10^4$  for  $\mathbf{X}_+$ , roughly four orders of magnitude, because  $\mathbf{x}_6$  is a near-copy of  $\mathbf{x}_1$  and the two columns are almost linearly dependent. The training RMSE barely moves, from 0.67637 to 0.67547. Adding a predictor can never increase the training RMSE, and here the improvement is negligible because  $\mathbf{x}_6$  carries essentially no information beyond  $\mathbf{x}_1$ .

The individual coefficients, however, are extreme:  $\hat{\beta}_{x_1} \approx 351.7$  and  $\hat{\beta}_{x_6} \approx -350.3$ , in place of a true effect of 1.4. They are enormous and of opposite sign, and they very nearly cancel. Their sum,  $\approx 1.4179$ , is almost exactly the coefficient 1.4172 that  $\mathbf{x}_1$  received in the well-conditioned fit. This is the first sign of what is really going on: the data determine the combined effect  $\beta_{x_1} + \beta_{x_6}$  well, and the individual pieces hardly at all.

#### (b) Refitting the perturbed response.

```
b_Xp_star <- qr.coef(qr(Xp), ystar)

data.frame(
  coefficient = c("x1", "x6"),
  fit_y       = b_Xp[c("x1", "x6")],
  fit_ystar    = b_Xp_star[c("x1", "x6")],
  change      = b_Xp_star[c("x1", "x6")] - b_Xp[c("x1", "x6")]
)
```

|    | coefficient | fit_y     | fit_ystar | change |
|----|-------------|-----------|-----------|--------|
| x1 | x1          | 351.6766  | 341.6766  | -10    |
| x6 | x6          | -350.2588 | -340.2588 | 10     |

```
fit_y      <- as.vector(Xp %*% b_Xp)
fit_ystar <- as.vector(Xp %*% b_Xp_star)

c(rms_difference = rmse_of(fit_ystar - fit_y),
  max_abs_difference = max(abs(fit_ystar - fit_y)))
```

```
      rms_difference max_abs_difference
      0.001015879      0.002572675
```

```
# every other coefficient is untouched
max(abs((b_Xp_star - b_Xp)[c("Intercept", "x2", "x3", "x4", "x5")]))
```

```
[1] 5.329071e-15
```

Perturbing the response by the tiny vector  $10^{-3}\mathbf{u}$  moves the coefficient of  $\mathbf{x}_1$  by exactly  $-10$  and the coefficient of  $\mathbf{x}_6$  by exactly  $+10$ . Every remaining coefficient is unchanged to within  $10^{-14}$ , which is rounding error.

The fitted values, by contrast, barely move at all: the root mean squared difference is  $1.02 \times 10^{-3}$  and the largest absolute difference is  $2.57 \times 10^{-3}$ . A change of 10 in two coefficients produces a change of about 0.001 in the predictions.

### (c) Why the coefficients move so much and the predictions so little.

The identity  $\mathbf{y}^* - \mathbf{y} = 10(\mathbf{x}_6 - \mathbf{x}_1)$  explains both facts at once, and it holds exactly here since  $\mathbf{x}_6 - \mathbf{x}_1 = 10^{-4}\mathbf{u}$  and  $\mathbf{y}^* - \mathbf{y} = 10^{-3}\mathbf{u}$ .

The perturbation lies **exactly in the column space of  $\mathbf{X}_+$** , because it is a linear combination of two of that matrix's own columns. Least squares is linear in the response, so the change in the coefficient vector is precisely the coefficient representation of the perturbation, namely  $-10$  on  $\mathbf{x}_1$  and  $+10$  on  $\mathbf{x}_6$  and zero elsewhere. This is not an approximation or an instability in the algorithm; it is an algebraic identity, which is why the observed changes are exactly  $-10.000000$  and  $+10.000000$ .

The fitted values move only by the perturbation itself. Since  $\mathbf{y}^* - \mathbf{y}$  already lies in the column space, projecting it changes nothing, so the fitted values shift by exactly  $10^{-3}\mathbf{u}$  — a vector whose root mean square is about  $10^{-3}$ . The reconciliation is that the direction  $\mathbf{x}_6 - \mathbf{x}_1 = 10^{-4}\mathbf{u}$  is extremely short. Moving a long way along a very short direction in coefficient space produces only a very small movement in prediction space. Enormous, opposite-signed coefficient changes cancel almost perfectly when multiplied back through the design matrix.

The lesson for interpretation is that when two predictors are nearly collinear, their individual coefficients are not meaningfully identified even though the fit as a whole is perfectly stable. The data pin down the combination  $\beta_{x_1} + \beta_{x_6}$ , which stayed at about 1.418 across both fits, while the difference between the two coefficients is nearly unconstrained: it can be moved almost arbitrarily at negligible cost in residual sum of squares. Reporting “the effect of  $\mathbf{x}_1$  holding  $\mathbf{x}_6$  fixed” is not meaningful when the data contain almost no observations in which  $\mathbf{x}_1$  moves and  $\mathbf{x}_6$  does not. Predictions remain trustworthy; separate

coefficient interpretations, standard errors, and significance tests for the individual collinear predictors do not. This is exactly the situation that motivates regularization and careful variable selection later in the course.

## Question 5 (Data preparation and summary statistics)

The file `data/data-manipulation.csv` is a small constructed data table containing an observation identifier, a group label, two numeric predictors, and a response. Some response values are missing. For analyses involving the response, use only observations with a recorded response. Do not replace a missing response with zero or a group mean.

- a. Read the data and report its dimensions, column types, number of duplicated identifiers, and number of missing values in each column. Create an analysis table containing only observations with a recorded response, and define

$$\text{feature\_sum} = x_1 + x_2 .$$

- b. For each group, report the number of observations, the mean response, and the mean of `feature_sum`. State clearly which observations these summaries describe.
- c. Sort the analysis table by response from largest to smallest and report the first three rows, including `observation_id`, `group`, `response`, and `feature_sum`. Add code checks verifying that the number of retained rows equals the number of nonmissing responses in the original data, that the analysis table has no missing response values, and that its identifiers are unique.

## Answer

### (a) Reading, inspecting, and building the analysis table.

```
library(dplyr)

raw <- read.csv("data/data-manipulation.csv", stringsAsFactors = FALSE)

dim(raw)                                # dimensions
```

```
[1] 24  5
```

```
sapply(raw, class)                      # column types
```

| observation_id | group       | x1        | x2        | response  |
|----------------|-------------|-----------|-----------|-----------|
| "character"    | "character" | "numeric" | "numeric" | "numeric" |

```
sum(duplicated(raw$observation_id))      # duplicated identifiers
```

```
[1] 0
```

```
colSums(is.na(raw))
```

# missing values per column

```
observation_id    group      x1      x2    response
              0          0          0          0          3
```

The file holds 24 observations on 5 columns. `observation_id` and `group` are character, while `x1`, `x2`, and `response` are numeric. No identifier is duplicated. The only missing values are 3 in `response`, belonging to `obs-04`, `obs-11`, and `obs-20` — one in each group.

```
analysis <- raw |>
  filter(!is.na(response)) |>
  mutate(feature_sum = x1 + x2)

nrow(analysis)
```

```
[1] 21
```

```
head(analysis)
```

|   | observation_id | group | x1  | x2  | response | feature_sum |
|---|----------------|-------|-----|-----|----------|-------------|
| 1 | obs-01         | A     | 1.2 | 2.0 | 5.36     | 3.2         |
| 2 | obs-02         | A     | 2.4 | 1.1 | 6.18     | 3.5         |
| 3 | obs-03         | A     | 0.8 | 2.8 | 4.62     | 3.6         |
| 4 | obs-05         | A     | 2.0 | 2.2 | 5.52     | 4.2         |
| 5 | obs-06         | A     | 1.7 | 1.8 | 5.64     | 3.5         |
| 6 | obs-07         | A     | 2.8 | 0.9 | 7.18     | 3.7         |

The analysis table keeps the 21 observations that have a recorded response and adds `feature_sum =  $x_1 + x_2$` . The three observations with a missing response are dropped rather than filled in. Substituting a zero would invent a response far below every value actually observed, and substituting a group mean would fabricate data at exactly the quantity being summarized, shrinking the apparent variability and biasing any later standard error. Since the response is what is being modeled, an observation without one carries no information about the relationship and is simply set aside.

## (b) Group summaries.

```
group_summary <- analysis |>
  group_by(group) |>
  summarise(
    n_observations = n(),
    mean_response = mean(response),
    mean_feature_sum = mean(feature_sum),
    .groups = "drop"
```

```
)
```

```
group_summary
```

```
# A tibble: 3 × 4
  group n_observations mean_response mean_feature_sum
  <chr>   <int>         <dbl>         <dbl>
1 A         7           5.6           3.6
2 B         7           7.60          4.93
3 C         7           8.00          6.59
```

| group | n | mean response | mean feature_sum |
|-------|---|---------------|------------------|
| A     | 7 | 5.600         | 3.600            |
| B     | 7 | 7.600         | 4.929            |
| C     | 7 | 8.000         | 6.586            |

These summaries describe **only the 21 observations that have a recorded response**, not all 24 rows of the original file. Each group began with 8 observations and lost exactly one to a missing response, so each row above summarizes 7 observations. The means of `feature_sum` are computed on that same restricted set, even though `x1` and `x2` are recorded for all 24 rows. Reporting a mean of `feature_sum` over all 24 observations alongside a mean response over 21 would describe two different sets of observations in the same table, so the restriction is applied consistently to both columns.

### (c) Sorting and verification.

```
top_three <- analysis |>
  arrange(desc(response)) |>
  select(observation_id, group, response, feature_sum) |>
  head(3)
```

```
top_three
```

```
  observation_id group response feature_sum
1         obs-22   C      9.24          6.6
2         obs-16   B      9.00          5.2
3         obs-14   B      8.66          5.0
```

| observation_id | group | response | feature_sum |
|----------------|-------|----------|-------------|
| obs-22         | C     | 9.24     | 6.6         |
| obs-16         | B     | 9.00     | 5.2         |
| obs-14         | B     | 8.66     | 5.0         |

```
# 1. retained rows equal the number of nonmissing responses in the original data
stopifnot(nrow(analysis) == sum(!is.na(raw$response)))

# 2. the analysis table contains no missing response
stopifnot(!any(is.na(analysis$response)))

# 3. identifiers in the analysis table are unique
stopifnot(!any(duplicated(analysis$observation_id)))

c(
  retained_rows      = nrow(analysis),
  nonmissing_responses = sum(!is.na(raw$response)),
  missing_in_analysis = sum(is.na(analysis$response)),
  duplicate_ids       = sum(duplicated(analysis$observation_id))
)
```

```
retained_rows nonmissing_responses missing_in_analysis
           21                21                0
duplicate_ids
           0
```

All three checks pass. The analysis table has 21 rows, matching the 21 nonmissing responses in the raw file exactly, contains no missing response, and has no repeated identifier. Using `stopifnot` rather than printing the comparisons means the document fails to render if any assumption is ever violated, so a silent data problem cannot slip through unnoticed on a later run.

## Question 6 (Can 39 million Fitbit records represent US adults?)

Patten et al. (2026) describe Fitbit data from 59,018 participants in the All of Us Research Program. The dataset spans 14 years and contains more than 39 million daily step records. Participants contributed data through one of two routes:

- Bring Your Own Device (BYOD): participants shared data from a Fitbit they already owned.
- Wearables Enhancing All of Us Research (WEAR): invited participants received a Fitbit at no cost.

In the general activity cohort, the BYOD and WEAR groups contained 32,035 and 22,474 participants, respectively. The BYOD value is listed first in each comparison below:

- 77.3% versus 55.1% reported being White;
- 6.1% versus 15.2% reported annual household income between \$10,000 and \$25,000; and
- median daily steps were 6,867 versus 5,797.

Suppose the target is the mean of the participant-specific average daily step counts among US adults during the study period, with each adult given equal weight. A researcher writes:

“This dataset contains more than 39 million daily Fitbit records. Therefore, the average of all recorded step counts should provide an accurate estimate of mean daily activity among US adults.”

- a. Are the 39 million daily records independent observations? What characteristics or behaviors could cause some participants to contribute more recorded days than others? Explain how averaging all recorded days would then weight participants unequally.
- b. BYOD participants reported a median of 6,867 daily steps, compared with 5,797 among WEAR participants. Does this comparison show that already owning a Fitbit causes people to walk more? Explain your answer and give at least one plausible alternative explanation based on how participants entered the two groups.
- c. A very large dataset can still produce a biased estimate of a population quantity when the people and observations entering the dataset are selected. Using this study, explain how selection of both participants and recorded days could make the observed data differ from the US adult population. Why might the naive average of all recorded daily step counts fail to estimate the stated target? You do not need to determine the direction of the bias.

## Source

Patten, T., Preble, E. A., Master, H., et al. (2026). The All of Us Research Program's wearables dataset. *Nature Medicine*, 32, 2302-2310.

## Answer

### (a) Are the 39 million daily records independent observations?

No. The records are **clustered within participants**. The 39 million daily records come from only 59,018 people, so on average each person contributes hundreds of days. Two step counts from the same person are far more similar to each other than two step counts from different people, because they share a commute, an occupation, a fitness level, a neighborhood, and a single device with its own calibration. Days from the same person are also serially correlated: an injury, a vacation, or a new job shifts many consecutive days together. The effective sample size is therefore closer to the order of tens of thousands of people than to 39 million independent observations, and treating  $N = 39,000,000$  as the sample size would understate every standard error dramatically.

Participants also differ systematically in **how many days they contribute**. Enrollment date and length of follow-up vary across a 14-year span. Adherence varies: some people wear the device daily for years, others lose interest within weeks, and abandoned devices stop producing records entirely. A device must be charged, worn, and synced for a day to appear at all. Device replacement costs money, so participants who can afford a new tracker when the old one fails keep contributing. Health matters too, since illness, injury, or hospitalization both reduces step counts and tends to interrupt wear.

Averaging all recorded days therefore **weights each participant in proportion to the number of days they contributed**, not equally. The stated target gives each US adult equal weight; the naive average gives each recorded *day* equal weight. A highly engaged participant with 2,000 recorded days counts 100 times as much as one with 20 days. If sustained device use goes together with being more active, which is plausible since motivated exercisers are the ones who keep wearing a fitness tracker, then the heavily weighted participants are also the more active ones, and the pooled average is pulled upward for reasons that have nothing to do with US adults.

### (b) Does the BYOD versus WEAR comparison show that owning a Fitbit causes more walking?

No. Participants were **not randomly assigned** to the two routes. They sorted themselves: BYOD participants are precisely those who had already chosen to buy a fitness tracker, while WEAR participants

received one because they did not have one. The comparison of 6,867 against 5,797 median daily steps therefore compares two different kinds of people, not the same people under two conditions.

The reported group characteristics make the confounding concrete. BYOD participants were 77.3% White against 55.1% among WEAR, and only 6.1% reported household income between \$10,000 and \$25,000 against 15.2% among WEAR. The groups differ substantially in socioeconomic composition, and income and related factors are associated with occupation, neighborhood walkability, leisure time, access to safe places to exercise, and baseline health, all of which affect daily step counts directly.

The most plausible alternative explanation is **reverse selection on the outcome**: people who already walk a lot are the people most likely to buy a fitness tracker in the first place. Buying a Fitbit is a consequence of an existing interest in physical activity, so the higher median among BYOD participants may simply reflect who chooses to own one. A second explanation is differential adherence: someone who spent their own money on a device, and who was already tracking activity before enrolling, is likely to wear it more consistently and especially on active days, while a participant handed a free device may wear it more sporadically. That difference alone can shift a median without any difference in true activity. A third is straightforward socioeconomic confounding through the income and demographic gaps noted above.

### **(c) How selection can bias an estimate no matter how large the dataset is.**

Selection operates at **two distinct levels** here, and both push the observed data away from the target population.

*Selection of participants.* The All of Us Research Program enrolls volunteers rather than drawing a probability sample of US adults. Reaching the wearables cohort requires still more filtering: a participant needs a smartphone and internet access, must consent to share continuous location-adjacent activity data, must be willing to wear a device daily, and in the BYOD case must already own a Fitbit. Each of these steps selects on characteristics related to activity, health, income, age, and technology comfort. The 77.3% versus 55.1% White composition and the income gaps confirm that the resulting cohort does not mirror the US adult population.

*Selection of recorded days.* Even for enrolled participants, days appear in the data only when the device was charged, worn, and synced. Days that are missing are unlikely to be missing at random. A person is less likely to wear a tracker while sick, injured, hospitalized, traveling, or during an unusually busy stretch, and precisely those days tend to have atypical step counts. The recorded days are therefore a filtered subset of a participant's actual days, plausibly tilted toward ordinary or active ones.

The naive average fails for three compounding reasons. It estimates a mean over the **wrong population**, since the cohort is not representative of US adults. It uses the **wrong weights**, averaging over days rather than people, so participants with long records dominate a target defined to weight each adult equally. And it averages over a **filtered subset of days** within each participant, so even a single participant's contribution need not equal their true average daily step count.

The essential point is that a large  $N$  reduces **variance**, not **bias**. As the number of records grows, the naive average converges ever more tightly around the mean of the population that actually entered the dataset, which is not the target. The standard error shrinks toward zero while the systematic gap stays exactly where it is, so the result is a very precise estimate of the wrong quantity, reported with narrow confidence intervals that convey false confidence. Increasing the record count from 39 million to 390 million would not help; it would only make the wrong answer look more certain. What is needed instead is a change

of method: weighting participants equally rather than weighting days, reweighting or poststratifying the cohort toward known US adult benchmarks, and modeling the mechanism by which days go missing. Determining the direction of the bias is not required, and in fact it is not obvious, since the participant selection and the day selection need not push the same way.